Hugging Face RAG with HyDE and Hybrid Search

Varshith Gandra | Technical report

Project and dataset

This project answers questions using official Python documentation. I implemented BM25, dense retrieval, score fusion, HyDE, and cross-encoder reranking. The saved results show the strengths and weaknesses of each method on a small documentation corpus; they are not a general benchmark.

The corpus is a fixed snapshot of eight Python 3.13 documentation pages, covering data structures, control flow, modules, errors, classes, input/output, collections, and itertools. Chunk sizes of 128 and 220 embedding-tokenizer tokens produced 616 and 330 chunks, respectively. Both used 24-token overlap and stayed within the embedding model input limit. Metadata includes source URL, document ID, chunk ID, offsets, title, version, and checksum.

I prepared 36 held-out test questions and 12 development questions. The test set has six questions for each type: exact keyword, paraphrase, multi-hop, long natural language, ambiguous, and unanswerable. Relevant chunk IDs come from verified evidence spans. Development MRR@10 selects alpha; no test answer is used for that choice. Ambiguous questions are scored against one stated interpretation, and relevance labels may miss other valid passages.

Implementation choices

Component	Implementation
Embedding and index	all-MiniLM-L6-v2; normalized 384-dimensional vectors; FAISS inner product
Lexical search	BM25Okapi on lowercase alphanumeric and underscore tokens
HyDE and answers	google/flan-t5-base; greedy CPU generation; no fine-tuning
Reranking	cross-encoder/ms-marco-MiniLM-L-6-v2 on the top 30 fused candidates

For HyDE, the generator writes a hypothetical document from the original question. Its embedding drives dense retrieval, while BM25 still uses the original question. The hypothetical text is never included as answer evidence. Fusion min-max normalizes each 30-candidate list, fills missing contributions with zero, and combines alpha times BM25 plus (1-alpha) times dense. Duplicate IDs are removed. Constant lists map to zero; min-max scaling remains sensitive to outliers.

The final answer prompt uses at most three retrieved chunks, shortened to fit a 480-token input budget. The program saves the exact excerpts. Source numbers from the model are mapped to chunk IDs; invalid IDs are removed. When the model omits citations, a separate context-source list is added for traceability, not as proof of support.

Measured test results

E1: BM25; E2: dense; E3: hybrid; E4: HyDE+dense; E5: HyDE+hybrid; E6: E5 plus reranking. The table shows completed 128-token runs with development-selected weights. All three weights (0.25, 0.50, 0.75) are reported in results.csv. Answer generation is evaluated only for the selected configurations.

Tokens	Method	Alpha	R@5	R@10	MRR@10	nDCG@10	Answer F1
128	E1	-	0.517	0.592	0.405	0.438	0.227
128	E2	-	0.450	0.633	0.413	0.421	0.234
128	E3	0.5	0.583	0.692	0.500	0.512	0.274
128	E4	-	0.433	0.617	0.385	0.414	0.270
128	E5	0.5	0.583	0.692	0.502	0.521	0.235
128	E6	0.5	0.592	0.775	0.539	0.572	0.378

Retrieval averages use the 30 answerable test questions. Recall divides retrieved relevant chunks by all labeled relevant chunks. MRR@10 is the reciprocal of the first relevant rank, or zero if missing from the top 10. nDCG@10 uses binary relevance. Answer F1 is token overlap against expected answers across all 36 questions; it does not measure factual faithfulness. Equivalent explanations can receive low F1.

Latency and memory at 128 tokens

Method	Retrieve, mean s	Generate, mean s	Shared peak, MiB
E1	0.002	2.315	1857
E2	0.010	2.382	1857
E3	0.012	2.535	1857
E4	1.430	2.841	1857
E5	1.432	2.587	1857
E6	2.102	2.667	1857

These are warm-model CPU measurements with four PyTorch threads. Query embeddings and HyDE outputs were cached across configurations; their measured preparation cost was charged back to each request. Downloads, model loading, and indexing are excluded. RSS includes all three models and caches, so it is a shared full-application peak footprint, not an isolated peak for each method. P95 times and index sizes are in the CSV. Linux high-water RSS was measured for the shared experiment process.

Answers to the evaluation questions

1. When did BM25 outperform dense retrieval?

At 128 tokens, BM25 had a higher reciprocal rank on 8 of 30 answerable questions. Exact API names can help lexical matching, although common words and code-heavy passages can also distract it. For "Which directory holds cached Python .pyc files?" the reciprocal ranks were 1.000 versus 0.250.

2. When did dense retrieval recover missed evidence?

Dense retrieval had a higher reciprocal rank on 12 questions. It can match related wording that does not share all the same keywords. For "Which exception signals the end of an iterator?" the reciprocal ranks were 0.167 versus 0.000.

3. Did HyDE improve paraphrased questions?

For the six paraphrase questions at 128 tokens, dense MRR@10 was 0.574, compared with 0.688 for HyDE+dense. Hybrid scored 0.663, compared with 0.867 for HyDE+hybrid. The measured comparison is more useful than assuming HyDE always helps: invented wording may move the embedding away from the right evidence.

4. Did HyDE introduce irrelevant information?

For the out-of-corpus administrator-password question, the hypothetical text began: The password for the Python documentation administrator account is:. This text is generated without evidence. Even plausible-looking technical text cannot establish a real password or a fact missing from the corpus. Inspect answers.jsonl for every hypothetical document.

5. Which alpha worked best?

Development-selected weights were 128 tokens: E3=0.5, E5=0.5. These maximize average development MRR@10, not every query type. The small development set makes the selection uncertain; per_question.csv supports checking the trade-offs by type.

6. Did reranking improve retrieval or only answers?

At 128 tokens, reranking changed Recall@10 from 0.692 to 0.775, MRR@10 from 0.502 to 0.539, and answer F1 from 0.235 to 0.378. It can reorder existing candidates but cannot recover evidence absent from its top-30 input.

Chunk size, missing answers, and remaining limits

7. How did chunk size affect retrieval and generation?

The runtime stopped during the 220-token evaluation after 16 BM25 answers. I compared those same 16 questions at both sizes, rather than comparing different question sets. This subset contains exact, paraphrase, and some multi-hop questions; it is not representative of all six query types. The full code can rerun all configurations.

Tokens	Questions	R@5	R@10	MRR@10	Answer F1
128	16	0.594	0.641	0.481	0.210
220	16	0.698	0.823	0.572	0.249

On this paired subset, larger chunks improved retrieval recall and answer overlap. They preserve more context but also increase the text competing for the fixed prompt budget. Relevance IDs are remapped at each size, so the number of relevant chunks changes. This result does not establish that 220 tokens is best for every method.

8. What happened when the answer was absent?

The six no-answer test questions have no relevant chunk IDs. E6 explicitly refused 83.3% of them at 128 tokens. The prompt alone is not a reliable abstention mechanism: a missing answer can still produce an unsupported response. These questions are excluded from retrieval averages but included in answer F1.

9. What were the latency and memory costs?

Mean retrieval time ranged from 0.002 to 2.102 seconds at 128 tokens. HyDE adds generation and embedding; reranking adds pairwise inference. The shared process peak was 1857.2 MiB, including all models and caches. Serialized BM25 and FAISS indexes used 1.24 MiB. These are warm CPU measurements and a shared application footprint, not isolated per-method memory costs. Timing was not preserved for the incomplete 220-token configuration.

10. What failure modes remain?

E6 produced valid model-generated source IDs in 13.9% of answers. A valid ID only proves that the chunk was supplied, not that it supports every claim. The fallback context-source list provides traceability, not verified citations. Other limits include incomplete relevance labels, ambiguity, clipped context, lexical noise, hallucinated HyDE terms, and weak generation. Token F1 can penalize correct paraphrases and miss factual errors.

Next steps are broader relevance judgments, independent claim-support checks, stronger abstention rules, and more evaluation questions. The saved 128-token results cover all six methods and all required query types. The smaller chunk-size comparison and interrupted second sweep are explicit limitations. No unexecuted results are presented as measurements.

Reproducibility and references

Run main.py --evaluate with the bundled corpus to reproduce the full sweep. README.md contains VS Code commands. CSVs contain metrics; answers.jsonl preserves the exact model context and responses. The demo prints all stages and creates demo.json. The architecture is supplied as architecture.svg.

Sources: Python 3.13 documentation and license (docs.python.org/3.13/); Hugging Face cards for all-MiniLM-L6-v2, google/flan-t5-base and ms-marco-MiniLM-L-6-v2; Sentence Transformers (sbert.net); FAISS; rank_bm25; Gao et al., HyDE, arXiv:2212.10496. Exact URLs are in README.md.